

برنامه‌نویسی پیشرفته — Java

دکتر خانمیرزا ، مهندس زمانیان | بهار ۱۴۰۵



سیستم مدیریت کتابخانه

library management system

Encapsulation · Inheritance · Regular Expressions · Collections

۱. معرفی پروژه

شما باید یک سیستم مدیریت کتابخانه (LMS) را با Java بسازید. این پروژه در طول ترم در سه فاز انجام می‌شود و هر فاز روی چیزهایی که در فاز قبلی ساخته‌اید ادامه می‌دهد. تصمیم‌هایی که الان می‌گیرید، در فازهای بعدی یا به شما کمک می‌کنند یا کار را سخت می‌کنند. این فایل فقط مربوط به فاز اول است. سیستم کتابخانه باید کتاب‌ها، مجله‌ها و منابع دیجیتالی را که اعضا می‌توانند امانت بگیرند مدیریت کند. همچنین باید رزروها، تاریخچه امانت و جریمه‌های دیرکرد را هم نگه دارد. همه این‌ها را باید با طراحی درست شی‌گرا مدل کنید.

فاز ۱ (همین فایل)	Encapsulation, Inheritance, Regular Expressions, Collections
فاز ۲	Polymorphism, Generics, Interfaces, Exception Handling
فاز ۳	Threads & Concurrency, JavaFX GUI

۲. دریافت ورودی و خروجی

برنامه شامل منوها و زیرمنوها است و باید به صورت تعاملی عمل کند. زمانی که برنامه اجرا می‌شود، امکانات و منوهای مربوط به هر مرحله را نمایش داده و منتظر دریافت اقدام از کاربر می‌ماند. پس از انتخاب کاربر، برنامه آن را پردازش کرده و اقدام مورد نظر کاربر را انجام می‌دهد یا در صورت نیاز کاربر وارد زیرمنوهای مربوط به آن بخش می‌شود. توجه کنید که در هر مرحله، باید امکان بازگشت به عقب وجود داشته باشد. این چرخه تا دریافت کلیدواژه خاصی (مثلاً quit) ادامه پیدا می‌کند.

زمانی که تعداد نتایج خروجی زیاد باشد نمایش همه نتایج به صورت یکجا، می‌تواند باعث سردرگم شدن کاربر و همچنین کاهش کارایی سیستم شود. برای حل این مشکل، نتایج به صورت محدود (به عنوان مثال ۱۰ نتیجه اول) باید نمایش داده‌شوند و می‌توان با گزینه‌هایی بین صفحه‌های نتیجه جابه‌جا شد یا به مرحله قبل بازگشت.

۳. مدیریت خطاها

یکی از مهم‌ترین ویژگی‌های یک نرم‌افزار خوب برخورد مناسب با خطاهاست. بنابراین برنامه باید قادر باشد به صورت مناسب با حالت‌های خطا برخورد کند. به عنوان مثال در صورتی که کاربر اقدامی تعریف نشده انتخاب کند، برنامه نباید متوقف شود و باید با پیغام خطای مناسب کاربر را از مشکل مطلع کند.

۴. نقش‌های سیستم

در این برنامه کاربران می‌توانند یکی از نقش‌های کاربر معمولی یا پشتیبان باشند. کاربر معمولی همان کاربر اصلی کتابخانه است که بین آیتم‌های کتابخانه جست‌وجو می‌کند و آیتم مورد نظر خود را قرض می‌گیرد. پشتیبان‌ها وظیفه رسیدگی به درخواست‌های کاربران و اضافه کردن محتوای جدید به کتابخانه را دارند.

۴.۱ نقش کاربر معمولی

کاربر معمولی به عنوان مهم‌ترین عضو برنامه نقش اساسی در پیشرفت و توسعه کتابخانه دارد! هر کاربر معمولی می‌تواند سه نقش داشته باشد که عبارتند از: استاد، دانشجو و میهمان.

تنها فرقی که بین این سه نقش وجود دارد ماکزیمم تعداد آیتم‌هایی است که قرض گرفته‌اند و هنوز پس نداده‌اند، به طوری که کاربران میهمان تنها می‌توانند ۲ آیتم قرض بگیرند و در صورتی که بخواهند آیتم بیشتری قرض بگیرند باید حداقل یک آیتمی که قبلاً قرض گرفته‌اند را پس دهند. این تعداد برای دانشجو و استاد به ترتیب ۱۰ و ۱۵ می‌باشد. کاربران معمولی در کتابخانه با ID (با فرمت مشخص -FAC، -STU یا -GST به همراه digits) خود شناسایی می‌شوند و در صورتی که از قبل حساب کاربری نداشته باشند می‌توانند در منوی شروع با وارد کردن اطلاعات خود (نام، نام خانوادگی، ایمیل، شماره تلفن همراه و رمز عبور) در سیستم ثبت‌نام کنند.

خطاها: شماره تلفن یا ایمیل کاربر از قبل در سامانه وجود داشته باشد، رمز عبور ضعیف باشد (رمز عبور باید حداقل ۸ کاراکتر باشد و شامل حداقل یک حرف بزرگ و کوچک، یک عدد و یک کاراکتر خاص باشد) و فرمت ایمیل یا شماره تلفن معتبر نباشد.

پس از آن کاربر می‌تواند با ایمیل (یا شماره تلفن) و رمز عبوری که هنگام ثبت‌نام انتخاب کرده است وارد برنامه شود.

خطاها: اطلاعات وارد شده اشتباه باشد و با هیچ کاربری همخوانی نداشته باشد.

در صورتی که به عنوان کاربر معمولی وارد سیستم شویم این امکانات را داریم:

- ♦ جست‌وجوی کتاب‌ها: کاربران برنامه می‌توانند بین تمام آیتم‌های برنامه جست‌وجو کنند تا نتایج مورد نظر خود را پیدا کنند. ساده‌ترین روش، جست‌وجو بر اساس نام آیتم است که کاربر نام کالای مورد نظر خود را وارد می‌کند و کتاب‌هایی که نام آن‌ها شامل عبارت وارد شده باشد به عنوان خروجی نمایش داده می‌شوند. نوع دوم این است کاربر می‌تواند بر اساس پارامترهای مختلفی فیلترینگ انجام داده تا نتایج به‌خصوص‌تری را بدست آورد. این فیلترینگ می‌تواند بر اساس نوع آیتم (Book, Magazine, Ebook و AudioBook)، کتگوری، تاریخ انتشار و تعداد صفحات انجام شود.
- ♦ قرض کتاب: هر بار که کاربر می‌خواهد آیتمی به امانت بگیرد، آیتم‌هایی که پیش از آن قرض کرده است توسط سیستم باید بررسی شوند و در صورتی که قرض جدید محدودیت‌های تعداد آیتمی که آن کاربر می‌تواند با توجه به نقش خود به امانت بگیرد نقض شود، یا کاربر پیش از این کتابی رزرو کرده باشد اما آن را تا قبل از تاریخ انقضا به کتابخانه برنگردانده باشد، کاربر نمی‌تواند آیتم جدیدی قرض کند و باید خطای مناسب به کاربر داده شود. در سناریو اول کاربر ابتدا باید حداقل یک آیتم از آیتم‌هایی که قبلاً به امانت گرفته است را پس دهد یا تاریخ انقضا آن را تمدید کند تا بتوان آیتم جدیدی برای او در سیستم ثبت کرد. و در سناریو دوم کاربر تنها زمانی می‌تواند آیتم جدیدی رزرو کند که جریمه مربوط به دیرکرد را پرداخت بکند (این جریمه بر اساس تعداد روزهای دیرکرد محاسبه می‌شود) که بعد از آن یا مهلت استفاده از آیتم را تمدید کند یا به کل آن را پس دهد.

- ♦ بررسی بدهی: در این بخش کاربر می‌تواند ببیند که آیا بدهی بابت پس ندادن آیتم، یا دیر پس دادن آن دارد یا خیر، و در صورتی که داشته باشد می‌تواند کیف پول خود را شارژ کرده و سپس جریمه خود را پرداخت کند.
- ♦ کیف پول: هر کاربر عادی در برنامه یک کیف پول مختص به خودش دارد که در این بخش می‌تواند موجودی فعلی‌اش را مشاهده کند و یا آن را به مبلغ مورد نظر خود شارژ کند. همچنین کاربر در این بخش توانایی مشاهده تراکنش‌های قبلی خود را دارد. تراکنش‌ها برای کاربر عادی شامل تراکنش‌های خرید و شارژ می‌باشند. کاربر می‌تواند با انتخاب هر کدام از تراکنش‌های دلخواه اطلاعات کامل آن را به همراه تاریخ انجامش مشاهده کند. همچنان کاربران می‌توانند تراکنش‌هایشان را بر اساس بازه‌های زمانی فیلتر کنند.
- ♦ مشاهده لیست کتاب‌های به امانت گرفته‌شده: در این بخش کاربر می‌تواند لیست آیتم‌هایی را که به امانت گرفته است را مشاهده کند. همچنین در این بخش کاربر باید این قابلیت را داشته باشد که به دلخواه خود تاریخ انقضا آیتمی را تمدید کرده یا آن را کلاً پس دهد.
- ♦ تنظیمات: در این بخش امکاناتی برای ویرایش حساب کاربری به کاربر داده می‌شود. کاربر می‌تواند در این بخش اطلاعات کاربری شامل ایمیل، شماره تلفن، نام، نام خانوادگی و یا رمز عبور خود را تغییر دهد.
خطاها: شماره تلفن یا ایمیل کاربر از قبل در سامانه وجود داشته‌باشد، رمز عبور ضعیف باشد (رمز عبور باید حداقل ۸ کاراکتر باشد و شامل حداقل یک حرف بزرگ و کوچک، یک عدد و یک کاراکتر خاص باشد) و فرمت ایمیل یا شماره تلفن معتبر نباشد.
- ♦ پشتیبانی: در این بخش کاربر می‌تواند مشکل یا درخواست پیگیری خود را نسبت به یکی از گزینه‌ها (درخواست کتاب و مشکل فنی) ثبت کند. به این صورت که ابتدا بخش مورد نظر را انتخاب می‌کند، سپس متن درخواست خود را نوشته و آن را ثبت می‌کند. علاوه بر آن کاربر می‌تواند در این بخش لیست درخواست‌های باز خود را مشاهده کند و با انتخاب هر یک از آن‌ها، جزئیاتی مانند متن درخواست، وضعیت درخواست (شامل ثبت‌شده و بسته‌شده) و پاسخ پشتیبان را مشاهده کند.

۴.۲ نقش پشتیبان

پشتیبان‌ها وظیفه رسیدگی و پاسخگویی به مشکلات و شکایات کاربران را دارند. کاربر پشتیبان امکان ثبت نام ندارد و برای نسخه اولیه کتابخانه، اطلاعات آن‌ها (شامل نام، نام کاربری و کلمه عبور) از قبل در سیستم وجود دارد و با استفاده از نام کاربری و کلمه عبور، می‌توانند وارد سیستم شوند.

خطاها: نام کاربری یا عبور اشتباه است.

کاربران پشتیبان پس از ورود امکانات زیر را مشاهده می‌کنند:

- ♦ افزودن آیتم: در این قسمت کاربر پشتیبان می‌تواند آیتم جدیدی به کتابخانه اضافه کند. در هنگام اضافه کردن آن باید نوع آیتم (Book, Magazine, Ebook و AudioBook) و اطلاعات مختص به آن نوع آیتم را نیز وارد کند.
- ♦ درخواست‌ها: در این بخش، پشتیبان می‌تواند لیست درخواست‌های پیگیری که توسط کاربران ثبت شده را مشاهده و با انتخاب هر یک از آن‌ها، جزئیات آن را مشاهده کند. توجه کنید که برای مشاهده لیست درخواست‌ها، می‌توان آن‌ها را بر اساس وضعیت و یا بخش مربوط فیلتر نمود. پشتیبان پس از رسیدگی به درخواست، پاسخی را برای کاربر ثبت نموده و وضعیت درخواست را بروزرسانی می‌کند.
- ♦ جریمه‌ها: در این بخش پشتیبان می‌تواند لیست مربوط به کاربرانی که موظف به پرداخت جریمه هستند را ببیند. این لیست از بیشترین تا کمترین مرتب شده است.
- ♦ تاریخچه: پشتیبان به واسطه این بخش می‌تواند ببیند که چه آیتم‌هایی توسط چه کسانی به امانت گرفته شده است به طوری که این لیست فقط ۱۰ آیتمی که به تازگی قرض گرفته شده‌اند را نمایش می‌دهد.

۵. آیتم‌های کتابخانه

- ♦ Book: کتاب یک آیتم فیزیکی کتابخانه است، که باید ID، تایتل، سال انتشار، کتگوری، وضعیت موجودی، تعداد صفحات، ISBN، تعداد نسخه‌های موجود و نام نویسنده را ذخیره کند.
- ♦ Magazine: مجله یک آیتم فیزیکی کتابخانه است، که باید ID، تایتل، سال انتشار، وضعیت موجودی، کتگوری، ISSN، تعداد نسخه‌های موجود و نحوه انتشار (ماهانه، هفتگی، فصلی) را ذخیره کند.
- ♦ Ebook و AudioBook: تمام آیتم‌های دیجیتالی یک فرمت مشخص (PDF, EPUB, MP3, AAC)، حجم فایل به مگابایت، کتگوری، لینک دانلود، ID، تایتل، سال انتشار، وضعیت موجودی، تعداد صفحات دارند. URL باید با <https://> شروع شوند.

۶. Encapsulation

Encapsulation یعنی حالت داخلی یک object را پنهان کنیم و فقط از طریق متدهای مشخص و کنترل‌شده به آن دسترسی بدهیم. در Java این کار با access modifier ها انجام می‌شود.

۶.۱ قوانینی که باید رعایت کنید

- ♦ تمام instance field ها در همه کلاس‌ها باید private باشند. هیچ استثنایی وجود ندارد.
- ♦ فیلدهایی که فقط برای subclass ها معنی دارند (و باید خوانده شوند اما دوباره public نشوند) باید private باشند و فقط یک protected getter داشته باشند — نه این‌که خود فیلد protected باشد.
- ♦ Public setter فقط وقتی مجاز است که کد بیرونی واقعاً نیاز داشته باشد یک مقدار را تغییر دهد. برای هر چیزی setter لازم نیست.
- ♦ Setter ها باید ورودی را validate کنند. Setter که هر مقداری را بدون بررسی قبول می‌کند، کارش را درست انجام نمی‌دهد.

۶.۲ مرجع Access Modifier

کاربرد	Access Modifier
همه instance field ها.	private
فقط وقتی که subclass باید مقداری را بخواند که متعلق به parent است و نباید public شود.	protected
constructor ها، getter های لازم، setter های توجیه‌پذیر، و متدهای رفتاری معنی‌دار.	public
چند helper کوچک که فقط داخل همان package استفاده می‌شوند. کم و با دقت استفاده شوند.	(default)

۷. Inheritance

Inheritance یعنی یک subclass بتواند field ها و رفتارهای parent را reuse کند و آن‌ها را خاص‌تر کند. وقتی از آن استفاده کنید که واقعاً یک رابطه is-a وجود داشته باشد. فقط برای share کردن code از inheritance استفاده نکنید — اگر رابطه is-a واقعی نیست، composition بهتر است.

۷.۱ قوانین نوشتن subclass ها

- ♦ super(): باید اولین statement در constructor هر subclass باشد. Java برای parent های abstract این را لازم دارد.

- ◆ @Override: برای هر متدی که parent را override می‌کند اجباری است. این annotation کمک می‌کند compiler اشتباهات تایپی و drift را پیدا کند.
- ◆ هیچ field ای را که قبلاً در parent وجود دارد دوباره تعریف نکنید. اگر parent یک title field دارد، child نباید آن را دوباره declare کند — آن را به ارث می‌برد.

۸. Regular Expressions

Regular expression ها به شما اجازه می‌دهند یک pattern تعریف کنید که یک string باید با آن تطابق داشته باشد. در Java از `java.util.regex.Pattern` و `Matcher` استفاده کنید. هر pattern را یکبار به صورت `static final compile` کنید — نه داخل متدی که چند بار اجرا می‌شود.

۸.۱ Validator های اجباری

یک کلاس utility به نام Validator بسازید (بدون نمونه، فقط متدهای static). این کلاس باید validator های زیر را پیاده‌سازی کند:

Validator	قانون
ISBN-13	با 979 یا 978 شروع شود و در ادامه آن ۱۰ رقم دیگر هم حضور پیدا کند
ISSN	فرمت: چهار digit، یک hyphen، سه digit، و بعد یک check digit یا کرکتر X. مثال: 1234-5678.

Email address	ایمیل باید local part درست داشته باشد (شامل حروف، اعداد، نقطه و برخی از کرکتر های ویژه) و سپس بعد از آن کرکتر @ همراه با یک domain درست باشد
Phone number	باید با یکی از +98، 0 و یا 98 شروع شده و سپس بعد از آن ۹ عدد بیاید
Member ID	باید با فرمت -FAC، -STU یا -GST به همراه دقیقاً ۶ digit باشد.
Item ID	باید با فرمت -EBK، -MAG، -BOK یا -AUD به همراه دقیقاً ۸ digit باشد.
Publish year	دقیقاً ۴ digit. باید بین ۱۴۵۰ (به میلادی) و سال جاری باشد.
Download URL	باید با https:// شروع شود

۹. نکات و چیزهایی که باید حواستان به آنها باشد

۹.۱ نکات طراحی

- ◆ قبل از این که هر کدی بنویسید، نمودار class خودتان را روی کاغذ بکشید. اگر رابطه ها روی کاغذ معنی ندهند، در کد هم معنی نخواهند داد.
- ◆ بین IS-A (inheritance) و HAS-A (composition) فرق بگذارید. یک Catalog یک Map از items دارد — خودش Map نیست.
- ◆ وقتی مطمئن نیستید یک field باید در parent باشد یا child، از خودتان بپرسید: آیا همه ی subclass ها به آن نیاز دارند؟ اگر نه، فقط باید در همان subclass باشد.
- ◆ منابع دیجیتال برای availability با منابع فیزیکی فرق دارند. در DigitalResource متد isAvailable() را متناسب با آن override کنید.

۹.۲ اشتباهات رایج

- ◆ همه چیز را public کردن. در ظاهر سریع‌تر است. اما وقتی بخواهید در فاز ۲ invariant اضافه کنید، دو برابر زمان از دست می‌دهید.
- ◆ ذخیره کردن داده‌ی مشتق‌شده به‌عنوان field. اگر یک مقدار همیشه از روی داده‌های دیگر قابل محاسبه است (مثل borrowedCount از روی یک لیست)، آن را جدا ذخیره نکنید. این کار باعث ناهمگام شدن می‌شود.
- ◆ استفاده از collection اشتباه. استفاده از ArrayList جایی که HashMap لازم است، $O(1)$ lookup شما را به $O(n)$ تبدیل می‌کند. ببینید چه عملیاتی را بیشتر انجام می‌دهید و بر همان اساس انتخاب کنید.
- ◆ نوشتن regex بدون تست کردن آن.
- ◆ cast کردن به یک type concrete. اگر مدام می‌نویسید (Book) item، احتمالاً design شما نیاز به بازبینی دارد.
- ◆ در نوشتن برنامه، لزوماً باید از مفاهیم شی‌گرایی و قواعد کد نویسی استفاده کنید. همچنین تمام تلاش خود را برای نوشتن کدی خوانا و دارای قابلیت نگهداری مناسب به کار ببرید. نوشتن کل برنامه در یک کلاس هیچ نمره‌ای را در بر نخواهد داشت. عدم رعایت قواعد کد نویسی، منجر به کسر نمره خواهد گردید. توصیه می‌شود مدلسازی اولیه را انجام دهید و قابلیت‌هایی که قرار است اضافه کنید را در ذهن داشته باشید. مدلسازی خود را می‌توانید با دستیار آموزشی بررسی کنید و اشکالات احتمالی را رفع کنید. سپس بعد از انجام مدلسازی اولیه (فقط کلاس‌ها و فیلدهای مربوط به ارتباط بین کلاس‌ها) اقدام به پیاده‌سازی قابلیت‌ها و منوها کنید. همچنین توجه کنید که استفاده از متدهای static و الگوی singleton باید حتماً دارای دلیل قانع‌کننده باشند که چرا از متدهای غیر static استفاده نشده است. در غیر این‌صورت نمره زیادی را از دست خواهید داد.

۹.۳ موارد امتیازی

- ♦ تست واحد: با رعایت اصول Unit Testing و با استفاده از کتابخانه JUnit برای برنامه خود تست واحد بنویسید.
- ♦ کنترل نسخه: با رعایت اصول Conventional Commit و کامیت‌های مستمر، از Git به صورت صحیح استفاده کنید.
- ♦ خروجی رنگی: منوها، خروجی‌ها و خطاها را به صورت رنگی و مرتب چاپ کنید (توجه کنید که در صورتی که افزودن این قابلیت باعث ناخوانایی و عدم رعایت اصول کد تمیز شود، باعث کسر نمره می‌شود).
- ♦ تولید گزارش: در قالب فایل HTML، جداول و نمودارهای مالی برای فروشندگان تولید کنید. خلاقیت بیشتر و گزارش‌های مفیدتر، نمره امتیازی بیشتری دارند.
- ♦ فرستادن ایمیل: در صورت صحیح بودن ایمیل وارد شده کاربر، یک ایمیل به کاربر فرستاده شده تا احراز هویت به صورت دو مرحله انجام شود (two-step verification).